

HANDOVER PDF EDITION

# Agent Documentation

Query, Ontology, and Taxonomy agents over Client Frontend, Corpus DB, and Semantic Control Kernel boundaries.

**Core rule:** agents share one browser shell, but not one authority. Query reads, Ontology writes lenses, and Taxonomy delegates governed workflows to the Kernel.

# Contents

|                                    |    |
|------------------------------------|----|
| First read: agent boundaries.....  | 3  |
| Source Of Truth.....               | 4  |
| Shared Agent Shell.....            | 5  |
| Shared Status UI.....              | 6  |
| Shared Source Viewer.....          | 7  |
| Configuration And Credentials..... | 8  |
| Query Agent.....                   | 9  |
| Ontology Agent.....                | 14 |
| Taxonomy Agent.....                | 21 |
| Agent Cooperation Model.....       | 27 |
| Handover Debug Map.....            | 28 |
| Non-Negotiable Boundaries.....     | 29 |

Recommended first pass for new maintainers: read sections 1-6 for the shared shell, then jump to the agent section that matches the behavior you are debugging.

# 1. First read: agent boundaries

The agents in The Ontology Machine are not generic chat boxes. They are local interaction surfaces over specific tool sets, database boundaries and Kernel contracts.

This matters for handover because most visible behavior looks like a normal LLM chat, while the actual system is split into three very different agent roles:

| Agent          | Main Job                                                                      | Mutation Boundary            |
|----------------|-------------------------------------------------------------------------------|------------------------------|
| Query Agent    | Read and explain the active Corpus DB                                         | read-only                    |
| Ontology Agent | Build and edit evidence-bound ontology lenses                                 | ontology/relation layer only |
| Taxonomy Agent | Drive Kernel-governed creation, ingest, merge, rebuild and recovery workflows | no direct DB mutation        |

All three live in the Client Frontend. They share the same browser shell, credential/config layer, chat session infrastructure and health UI, but they do not have the same tools or authority.

## 2. Source Of Truth

The active agent implementation lives in:

- `Client Frontend/client_frontend/min_agent/`
- `Client Frontend/client_frontend/ontology_agent/`
- `Client Frontend/client_frontend/pipeline_agent/`
- `Client Frontend/client_frontend/http/application_context.js`
- `Client Frontend/client_frontend/http/application_agents.js`
- `Client Frontend/client_frontend/http/chat_routes.js`
- `Client Frontend/client_frontend/browser/main_app/`
- `Client Frontend/client_frontend/browser/api/factory.ts`

The user-facing name is `Taxonomy Agent`. The code path still uses older `pipeline_agent`, `pipeline-manager` and `Pipeline Manager` names in several places. Treat those names as implementation aliases for the same Frontend agent surface unless a file explicitly talks about a different owner module.

## 3. Shared Agent Shell

The browser app exposes one main chat surface with three agent modes:



The browser state models those modes as:



The `pipeline` browser state is the Taxonomy Agent. It kept the historical Pipeline Manager name because the Kernel integration originally grew out of that route.

Each mode has its own chat endpoint:

| Agent          | Chat Endpoint                                   |
|----------------|-------------------------------------------------|
| Query Agent    | POST <code>/api/v2/chat</code>                  |
| Ontology Agent | POST <code>/api/v2/ontology-agent/chat</code>   |
| Taxonomy Agent | POST <code>/api/v2/pipeline-manager/chat</code> |

History routes are also separate. This avoids mixing Query answers, ontology editing conversations and Kernel workflow chats into one session store.

Startup creates all available agents from the same loaded application context:

1. Load config and secrets.
2. Resolve the active Corpus DB path.
3. Load Frontend policy and prompt overrides.
4. Create the Query Agent if the DB is valid.
5. Create the Ontology Agent if the DB is valid and the Ontology Machine root can be resolved.
6. Create the Taxonomy Agent if the Kernel/MCP side can be started.
7. Create separate chat stores and session managers per agent family.

If the configured DB cannot be opened as SQLite or does not contain a `documents` table, Query and Ontology agents are created as unavailable agents. The browser then receives a normal HTTP error instead of a half-working agent that fails later inside a tool call.

## 4. Shared Status UI

The main window shows agent readiness from `/api/v2/health`.

The visible status indicators include:

- LLM readiness
- embedding readiness where relevant
- OAuth/credential state where relevant
- Taxonomy/Kernel workflow state
- Base Graph availability
- Ontology lens count and active primary lens

The Base Graph and lens badges are computed from the active Corpus DB. Source documents/pages or structural units prove Base Graph availability. The `ontology_lenses` and `ontology_activation` tables prove how many lenses exist and which one is primary.

### Important handover point

the UI does not ask the Query Agent whether the Base Graph exists. It asks the DB through the Frontend repository layer. That keeps the health indicator deterministic.

## 5. Shared Source Viewer

The Query and Ontology modes share the source/page viewer surface.

Agent answers may contain inline references such as [1] or mention file names directly. The browser renderer turns resolvable references into source buttons. The source panel then opens the corresponding source cards and page image viewer.

Page images are resolved through:

1. `document_page_images` inside the Corpus DB.
2. Artifact-level `Documents/page_images/...` files when needed.

The DB image table is the query/evidence link surface. The artifact folder is the rebuild surface for the Corpus Builder.

Source display is deliberately separate from answer text. The answer is what the model says. The source column is what the current turn could resolve from actual tool output and DB rows.

That separation is useful because it can expose two important failure modes:

- a source named by the model was not touched or resolved in the current turn
- the model leaked a source from previous context into a new answer

When a source mention cannot be matched to the resolved source list, the UI can mark it as suspicious. That does not automatically prove the answer is wrong, but it tells the user that the cited source needs a second look.

## 6. Configuration And Credentials

The Config app exposes:

- setup and path configuration
- model settings
- Query Agent prompt
- Ontology Agent prompt
- advanced Frontend policy
- credentials and OAuth helpers

The active Query/Ontology DB path is stored in Frontend config. Query and Ontology tools do not ask the model to choose a DB path. They operate against the configured active Corpus DB.

The Taxonomy Agent is different. It talks to the Semantic Control Kernel through the local MCP bridge. Paths, target artifact trees, confirmations, resume choices and recovery details are collected through Kernel-owned dialogs or Kernel state, not through model-authored free text.

# 7. Query Agent

The Query Agent is the read-only corpus analyst.

It answers questions over the active Corpus DB. It can inspect raw and normalized material, source-document grouping, promotions, extracted fields, rows, evidence atoms, page images, Base Graph relations and ontology lenses.

It cannot write to the Corpus DB.

## User-Facing Surface

The user selects the Query tab in the Client Frontend and asks questions in normal language.

The answer should be grounded in the active DB and accompanied by source cards where the answer touched source documents or pages.

Typical questions:

- "What documents are in this corpus?"
- "Which invoices mention delayed shipping?"
- "Compare the active ontology lenses."
- "Show me the evidence for this field."
- "Which stories have a bitter ending?"

## Prompt Assembly

The Query Agent prompt is assembled from:

- editable `frontend_policy.min_agent.prompt`
- schema summary
- corpus size summary
- source-document guidance
- ontology lens guidance
- path hiding rules

The default prompt lives in:

```
Client Frontend/client_frontend/frontend_policy/defaults.js
```

The runtime prompt assembly lives in:

The key prompt rule is that ontology lenses are part of the DB meaning, not a special mode the user must explicitly invoke. If active lenses exist, the Query Agent should consider them during overview and detail answers. If an active correction, audit or review lens contradicts materialized facts, the Query Agent should surface that contradiction while keeping base facts and lens claims separate.

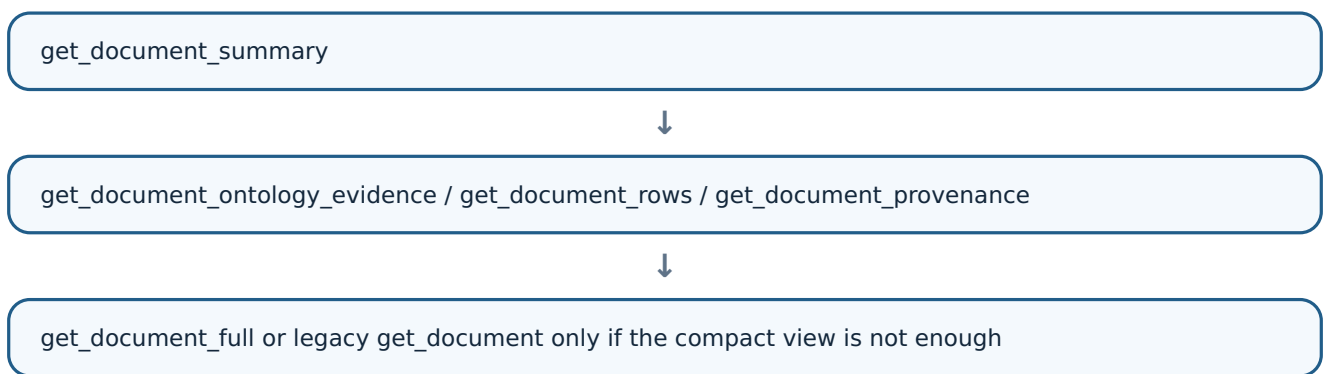
## Tools

The Query Agent exposes fifteen read-only tools.

| Tool                                        | Purpose                                                                                                                                                            |
|---------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>sql_query</code>                      | Run one read-only <code>SELECT</code> or <code>WITH</code> query against the active DB.                                                                            |
| <code>get_document_summary</code>           | Return compact document identity, source-document context, active promotions, structural hints and short excerpts.                                                 |
| <code>get_document_ontology_evidence</code> | Return compact ontology-facing evidence: source-document classifications, promotions, selected fields/rows, structural units, evidence atoms and bounded excerpts. |
| <code>get_document_rows</code>              | Return a row-focused view for tables, line items, orders, invoices, shipments and similar row-level checks.                                                        |
| <code>get_document_provenance</code>        | Return document-level provenance material when the exact target slot is not known yet.                                                                             |
| <code>get_document_full</code>              | Return the explicit full document inspection bundle as a last escalation step.                                                                                     |
| <code>get_document</code>                   | Legacy/full document read equivalent to the expensive full inspection bundle.                                                                                      |
| <code>get_provenance</code>                 | Resolve provenance for a field, row, promotion, candidate or evidence target.                                                                                      |
| <code>semantic_search</code>                | Search the corpus using vectors where available and lexical fallback where needed.                                                                                 |
| <code>database_coverage_snapshot</code>     | Return compact coverage counts and DB health signals.                                                                                                              |
| <code>list_source_documents</code>          | List deterministic source-document groups created by Base Graph mining.                                                                                            |
| <code>get_source_document</code>            | Read one source-document group and its pages.                                                                                                                      |

| Tool                              | Purpose                                                                              |
|-----------------------------------|--------------------------------------------------------------------------------------|
| <code>list_ontology_lenses</code> | List ontology lenses and activation state.                                           |
| <code>get_ontology_lens</code>    | Read one ontology lens with its terms, nodes, edges, assertions and classifications. |
| <code>workbench</code>            | Run a constrained read-only Python or PowerShell inspection task.                    |

The `get_document_*` tools are deliberately flat. They all go through the same repository owner path, but they give the model a clear escalation ladder instead of one huge document blob with many optional parameters:



This keeps source-aware document reads available while reducing accidental token burn on large corpora.

The normal tool loop is capped at 16 rounds. If the model keeps calling tools without producing a final answer, the workflow fails with a tool-round error.

## Read-Only Boundary

The Query Agent opens SQLite in read-only mode.

Its SQL policy allows only one statement and only `SELECT` or `WITH` queries. Mutating keywords such as `insert`, `update`, `delete`, `drop`, `alter`, `create`, `attach` and `pragma` are blocked.

The workbench is also constrained. It is meant for compact local read analysis, not as a general shell. It blocks writes, network access, process launch, UNC paths, generic shelling out and path traversal outside the active corpus/config scope.

## Search Behavior

`semantic_search` tries vector search first.

The preferred surface is:

embedding\_chunks

Older or coarser DBs may still expose:

embeddings

If vector search is unavailable or fails, the Query Agent falls back to lexical search over materialized text surfaces such as promotions, free text, metadata, fields and rows.

Embedding failure is therefore not a total Query Agent failure. It reduces retrieval quality and semantic reach, but the DB can still be inspected through SQL, document tools, coverage tools and lexical search.

## Source Reconciliation

The Query Agent collects source candidates from tool results, document rows, source-document reads, fallback text hints, `get_document_*` views and provenance reads. Before returning the answer, the server dedupes and normalizes sources into a public source payload.

The source column should be read as:

### **Read the source column as:**

sources the current turn could resolve from tool-visible corpus data

It should not be read as:

### **Do not read it as:**

all sources that would prove the answer beyond doubt

That difference is intentional. It leaves room for the user to see when the agent's prose and the tool-visible source trail diverge.

## Expected Output

A good Query Agent answer:

- answers the user question directly
- distinguishes facts from interpretation

- uses source-document grouping for multi-page documents
- mentions ontology lens interpretations when relevant
- surfaces correction/audit/review lens contradictions
- includes source-backed statements where the answer depends on corpus evidence
- does not expose absolute local paths unless the user explicitly needs them

## Failure Modes

| Failure                  | Meaning                                                                 |
|--------------------------|-------------------------------------------------------------------------|
| DB unavailable           | Config path is missing, invalid or not a compatible Corpus DB.          |
| SQL rejected             | The generated SQL was not a single read-only query.                     |
| Too many tool rounds     | The agent never returned a final answer within the round cap.           |
| Embeddings unavailable   | Semantic search degraded to lexical/SQL/document tools.                 |
| No new sources           | The answer produced no resolvable source trail for this turn.           |
| Suspicious source marker | The answer referenced a source not matched to current resolved sources. |

## 8. Ontology Agent

The Ontology Agent is the Knowledge Mining Layer over the active Corpus DB.

It can read the corpus broadly like the Query Agent, but it can write only into the ontology/relation layer through controlled tools. It does not rewrite the materialized corpus truth.

### Central design point

The Ontology Agent writes above the materialized corpus truth instead of rewriting it.

The Corpus Builder creates the evidence-bearing base: documents, page images, payloads, normalized data, extracted fields/rows, promotions, evidence atoms, embeddings, source identity and the deterministic Base Graph. The Ontology Agent works above that layer by creating lenses.

A lens can represent:

- a narrative reading
- a domain model
- a reviewer's judgement
- a correction or audit layer
- a risk model
- a character model
- a peer-review perspective
- a topic taxonomy over already materialized facts

The written result remains evidence-bound, versioned and computable.

## User-Facing Surface

The user selects the Ontology tab in the Client Frontend.

The Ontology Agent should help the user turn fuzzy intent into explicit semantic structure. It should also explain ontology basics when the user's request is ambiguous. The user should not need to already know what terms, nodes, edges, assertions, evidence links or lens activation mean.

Typical requests:

- "Build a story-arc lens over these stories."

- "Create a correction lens for bad invoice classifications."
- "Compare the cynical lens with the structural lens."
- "Run the Base Graph first."
- "Add evidence to this claim."
- "Make a reviewer lens for reviewer A."

## Prompt And Interaction Contract

The default Ontology Agent prompt lives in:

```
Client Frontend/client_frontend/ontology_agent/prompt_sections.js
```

Runtime policy assembly lives in:

```
Client Frontend/client_frontend/ontology_agent/policy.js
```

The prompt frames the agent as a local ontology engineer. It is expected to:

- inspect before editing
- ask for intent only when the missing decision cannot be inferred safely
- explain ontology concepts in practical terms
- translate fuzzy user language into terms, nodes, edges, assertions and evidence
- create small validated batches
- mention validation and embedding status after edits
- never hide uncertainty

The agent must not treat ontology creation as a single giant prompt over the whole DB. It should mine by reading, writing bounded patches, validating and continuing.

## Read Tools

The Ontology Agent inherits a filtered read subset from the Query Agent:

| Tool                           | Purpose                                                                        |
|--------------------------------|--------------------------------------------------------------------------------|
| sql_query                      | Read-only DB inspection.                                                       |
| get_document_summary           | Read compact document identity, source-document context and active promotions. |
| get_document_ontology_evidence | Read compact ontology-facing evidence for lens design and evidence-link work.  |

| Tool                                    | Purpose                                                                              |
|-----------------------------------------|--------------------------------------------------------------------------------------|
| <code>get_document_rows</code>          | Read row-level material without pulling the whole document bundle.                   |
| <code>get_document_provenance</code>    | Read document-level provenance material when the exact target slot is not known yet. |
| <code>get_document_full</code>          | Read the full document bundle only as a last escalation step.                        |
| <code>get_document</code>               | Legacy/full document read.                                                           |
| <code>get_provenance</code>             | Resolve evidence/provenance targets.                                                 |
| <code>semantic_search</code>            | Retrieve relevant corpus material.                                                   |
| <code>database_coverage_snapshot</code> | Inspect coverage and DB state.                                                       |
| <code>list_source_documents</code>      | Inspect Base Graph source-document groups.                                           |
| <code>get_source_document</code>        | Inspect grouped source documents and pages.                                          |
| <code>list_ontology_lenses</code>       | Inspect existing lenses.                                                             |
| <code>get_ontology_lens</code>          | Inspect a lens in detail.                                                            |

The Ontology Agent does not inherit the Query Agent `workbench`.

For ontology work, the intended read path is progressive. Start with `get_document_summary`, escalate to `get_document_ontology_evidence` for lens construction, use `get_document_rows` for table-like material, use `get_document_provenance` for document-level proof surfaces, and pull the full document only when the compact views do not answer the question. This does not make the agent less capable; it prevents it from drowning itself in JSON it did not actually need.

## Write Tools

The Ontology Agent adds two tools:

| Tool                               | Purpose                                                                                                                    |
|------------------------------------|----------------------------------------------------------------------------------------------------------------------------|
| <code>basic_relation_mining</code> | Run the deterministic Base Graph mining function over the active configured DB.                                            |
| <code>sql_batch_execute</code>     | Apply a bounded ontology/relation-layer SQL patch with preflight, transaction, validation, embedding refresh and edit log. |

`basic_relation_mining` accepts only `dry_run`.

`sql_batch_execute` accepts:

- edit summary
- optional `ontology_id`
- up to 50 SQL statements

The model does not pass a DB path. The tool always works on the active DB configured in the Client Frontend.

## Deterministic Base Graph

`basic_relation_mining` is deterministic. It does not involve an LLM.

The Client Frontend calls bundled Python, enters the Semantic Control Kernel ontology primitive, and then the Corpus Builder performs the owner mutation.

The operation builds or refreshes:

- source-document groups
- source-document pages
- deterministic source-document classifications
- Base Graph relations
- structural units
- structural unit relations

It uses materialized DB fields such as `source_document_id`, source URI and page indexes. It should not infer a grand semantic ontology. It creates the base structure needed so agents and users can stop treating page rows as unconnected fragments.

## SQL Write Boundary

The SQL write policy allows ontology/source relation layer writes only.

The intended write surface is:

- `ontology_*`
- `relations`
- `entity_relations`
- `source_documents`
- `source_document_pages`
- `source_document_classifications`

The semantic boundary is stricter than the table allowlist:

- the agent must not overwrite `documents`
- the agent must not rewrite payloads
- the agent must not rewrite extracted fields or rows
- the agent must not rewrite promotions
- the agent must not write deterministic `base` or `semantic_release` classifications
- ontology-authored classifications must use `classification_scope='ontology'`

This is why the Ontology Agent can create a correction lens without modifying the base fact it disagrees with. The contradiction remains explicit and queryable.

## Write Discipline

Ontology writes must be boringly explicit.

The agent is expected to provide stable IDs for:

- ontology lenses
- ontology terms
- ontology nodes
- ontology edges
- ontology assertions
- ontology evidence links
- ontology embedding chunks where applicable

It must insert parent records before child records. It must use valid JSON defaults such as `{}` where non-null JSON columns require them. It must not use `active` as a lens status because lens status values are `draft`, `ready` and `archived`; activation is represented separately.

Edges are node-to-node. Terms are vocabulary/typing structures, not edge endpoints.

Evidence links need their own stable `evidence_link_id`.

## Preflight And Repair Loop

Before a batch is executed, preflight checks:

- target table exists
- columns are known
- required NOT NULL values are present
- stable IDs are present

- parent references exist or are created earlier in the same batch
- edge endpoints are nodes in the same lens
- evidence targets exist
- embedding targets have valid object IDs
- ontology classifications use the ontology scope
- term/node mistakes are caught before SQLite foreign keys fire

If preflight returns a repairable failure, the workflow does not immediately throw the raw error to the user. It injects an internal repair instruction into the same model call and gives the agent up to three repair attempts.

The user should only see the failure after the repair budget is exhausted or when the missing decision is genuinely a user decision.

## Transaction, Validation And Embeddings

Accepted `sql_batch_execute` calls run in this order:

1. Normalize and compile statements.
2. Run write preflight.
3. Snapshot affected ontology tables.
4. Start `BEGIN IMMEDIATE`.
5. Execute all statements.
6. Mark affected ontologies dirty.
7. Commit.
8. Run Kernel ontology patch validation.
9. Refresh ontology embeddings if validation passes.
10. Write `ontology_edit_log`.

SQLite errors before commit roll back the batch.

Kernel validation opens the DB read-only and checks Base Graph consistency, ontology object invariants, JSON payloads, references, evidence links, embedding targets and activation rules.

If Kernel validation cannot run, the tool returns a warning. If validation runs and fails after commit, the write is not silently hidden. The result is returned as failed validation and the edit log records the failure. Embedding refresh is skipped or marked unavailable when the validated ontology state cannot be embedded safely.

## Expected Output

A good Ontology Agent answer:

- says what was read
- says what was written
- says which lens was created, updated or activated
- separates base facts from lens claims
- names unresolved ambiguity
- reports validation status
- reports embedding status when relevant
- does not ask the user to approve mechanical repair steps it can do itself

## Failure Modes

| Failure                 | Meaning                                                                      |
|-------------------------|------------------------------------------------------------------------------|
| DB unavailable          | The configured active DB is missing or invalid.                              |
| Write policy rejection  | SQL targeted a non-allowlisted table or unsafe operation.                    |
| Preflight failure       | The batch is structurally wrong before transaction.                          |
| Repair budget exhausted | The model failed to correct a repairable SQL batch within three attempts.    |
| SQLite rollback         | A transaction-time SQL error occurred before commit.                         |
| Validation failure      | Rows were written but failed Kernel ontology validation.                     |
| Embedding unavailable   | The ontology exists but semantic retrieval over ontology chunks is degraded. |

## 9. Taxonomy Agent

The Taxonomy Agent is the user-facing workflow agent for the Semantic Control Kernel.

It does not manually build taxonomies in chat and it does not directly mutate Corpus DB rows. It selects Kernel tools, explains Kernel state, follows Kernel dialogs and lets owner modules perform the work.

The code path is still named `pipeline_agent`.

### User-Facing Surface

The user selects the Taxonomy tab in the Client Frontend.

The user can either type a request or use the workflow dropdown. The dropdown shows the normal workflow starts a user is expected to trigger directly. It does not show every internal support tool.

The Taxonomy Agent also owns the visible Kernel workflow UI:

- workflow dropdown
- Kernel dialog modal
- progress panel
- abort button
- Kernel Reset button
- final notice messages

### Permanent Kernel Tool Surface

The Taxonomy Agent sees sixteen permanent Kernel-facing tools.

| Category          | Tool                                                             |
|-------------------|------------------------------------------------------------------|
| Empty DB creation | <code>empty_database_no_semantic_release</code>                  |
| Empty DB creation | <code>empty_database_default_taxonomy_no_projections</code>      |
| Empty DB creation | <code>empty_database_default_taxonomy_default_projections</code> |
| Empty DB creation | <code>empty_database_default_taxonomy_custom_projections</code>  |
| Empty DB creation | <code>empty_database_custom_taxonomy_no_projections</code>       |
| Empty DB creation | <code>empty_database_custom_taxonomy_custom_projections</code>   |

| Category           | Tool                                            |
|--------------------|-------------------------------------------------|
| Operation          | <code>manual_pipeline_run</code>                |
| Operation          | <code>database_merge_additive_only</code>       |
| Operation          | <code>database_rebuild_from_artifacts</code>    |
| Continuation/reset | <code>create_custom_taxonomy_path</code>        |
| Continuation/reset | <code>create_custom_projection_path</code>      |
| Continuation/reset | <code>reset_database</code>                     |
| Support            | <code>kernel_status</code>                      |
| Support            | <code>kernel_resume_state</code>                |
| Support            | <code>kernel_continue_resumable_workflow</code> |
| Support            | <code>kernel_cancel_active_run</code>           |

Most tool schemas are intentionally empty. The model should call the tool that matches the user's intent and let the Kernel ask for paths, confirmations or selections through dialogs.

The only permanent tool with a normal argument is:

```
kernel_continue_resumable_workflow(resume_option_ref)
```

The value is an opaque Frontend/Kernel-provided reference. The model must not invent raw resume IDs.

## Visible Workflow Dropdown

The browser dropdown exposes twelve workflow starts:

- the six empty database creation routes
- `manual_pipeline_run`
- `database_merge_additive_only`
- `database_rebuild_from_artifacts`
- `create_custom_taxonomy_path`
- `create_custom_projection_path`
- `reset_database`

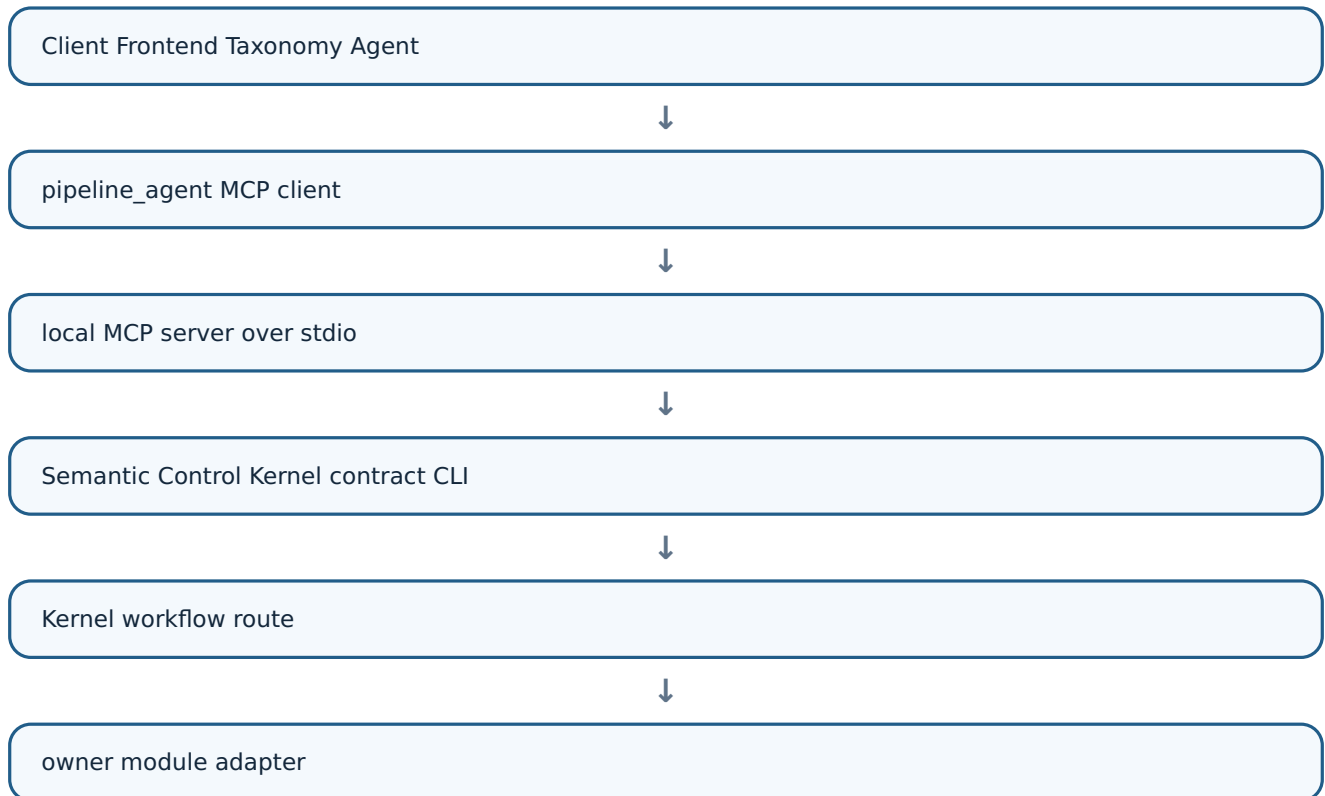
Selecting an entry does not bypass the agent. It sends a chat command telling the Taxonomy Agent to call the matching Kernel tool.

This keeps the UI discoverable while preserving the agent/Kernel flow.

## MCP Bridge

The Taxonomy Agent reaches the Kernel through the local MCP bridge.

Flow:



The Client Frontend starts MCP with a host bridge token. Agent-visible tools are listed through normal MCP `tools/list`. Host-only tools are hidden from the model and used by the Frontend to poll events, answer dialogs and inspect event-scoped tool definitions.

Host-only bridge tools include:

- `kernel_list_client_frontend_events`
- `kernel_submit_user_interaction_response`
- `kernel_cancel_user_interaction`
- `kernel_list_event_scoped_tool_definitions`

### Bridge boundary

The token check matters. Without it, a normal agent-facing MCP surface could see or call host-only bridge controls that should belong to the Client Frontend.

## Kernel Dialog Ownership

The Taxonomy Agent prompt explicitly tells the model not to collect Kernel-owned values in chat.

Kernel-owned values include:

- artifact tree paths
- DB paths
- release selections
- file/folder selections
- confirmations
- pending interaction IDs
- recovery IDs
- resume details
- raw resume IDs

If the Kernel needs one of those values, it emits a pending interaction event. The Frontend renders a dialog. The user answers the dialog. The Frontend sends that answer back through the host-only bridge. The model should explain what is happening, not replace the dialog protocol.

## Progress And Final Notices

Kernel workflows are long-running. The Frontend polls Kernel events and updates the progress panel with:

- current step
- progress event text
- pending interaction state
- recovery state
- blocker state
- final mirror events

Final notices are Kernel mirror events with `response_mode: explain_now`. The Taxonomy Agent receives those events and produces a user-facing explanation without calling more tools for that auto-explanation turn.

That final notice is the difference between "a background process stopped" and "the workflow finished or blocked with this exact result."

## Event-Scoped Recovery Tools

The permanent sixteen tools are not the whole Kernel recovery universe.

The Kernel can also expose event-scoped recovery tools for a specific blocked or recoverable event. Those tools are not stable public workflow starts. They are contextual options generated by current Kernel state.

For documentation and handover, treat the implementation in:

```
08 - Semantic Control Kernel/semantic_control_kernel/surface/event_scoped_tools.py
```

as the current source of truth for event-scoped recovery behavior.

## Expected Output

A good Taxonomy Agent answer:

- identifies the intended Kernel workflow
- calls the matching Kernel tool rather than inventing steps
- explains pending Kernel dialogs in plain language
- reports active, blocked, cancelled, resumable or completed state
- summarizes final notices after workflows complete
- tells the user when a blocker is an owner error, missing release, invalid target, timeout or recovery decision
- does not pretend that a Kernel workflow completed before the final notice

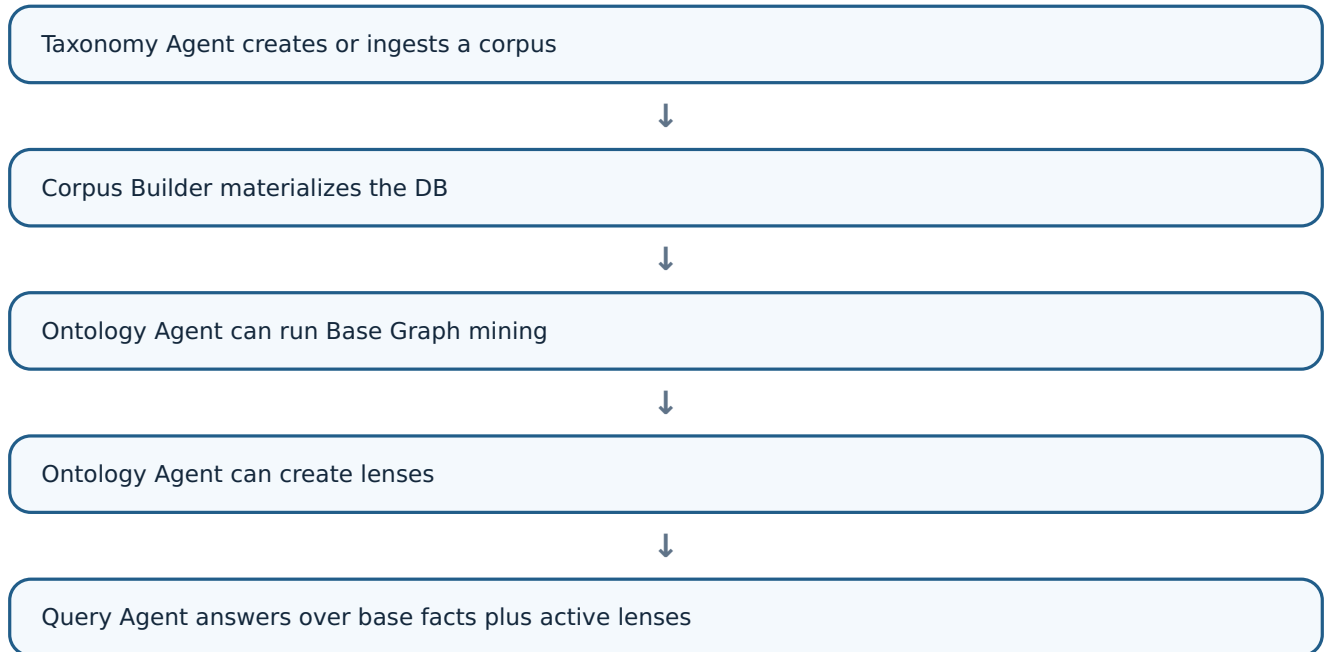
## Failure Modes

| Failure              | Meaning                                                             |
|----------------------|---------------------------------------------------------------------|
| MCP unavailable      | The local MCP bridge could not start or list tools.                 |
| Tool missing         | Kernel tool inventory and Frontend expected tool surface disagree.  |
| Pending dialog       | The Kernel is waiting for a user choice through the dialog UI.      |
| Owner error          | A module adapter failed inside the owner module.                    |
| Workflow blocked     | The Kernel route reached a blocker and wrote a blocked state.       |
| Resumable state      | The Kernel has opaque resume options the user may choose.           |
| Final notice missing | Workflow state finished but no mirror explanation reached the chat. |

# 10. Agent Cooperation Model

The three agents are meant to cooperate through the DB and Kernel, not through hidden shared chat memory.

Normal flow:



The Query Agent is the read surface.

The Ontology Agent is the post-materialized knowledge mining surface.

The Taxonomy Agent is the Kernel workflow surface.

They can all talk about the same active corpus, but they do not own the same state.

# 11. Handover Debug Map

Use this map when an agent behavior looks wrong.

| Symptom                                       | Start Here                                                                                      |
|-----------------------------------------------|-------------------------------------------------------------------------------------------------|
| Query answer misses documents                 | <code>Client Frontend/client_frontend/min_agent/</code> and DB read-surface views               |
| Query source looks hallucinated               | source reconciliation in <code>min_agent/source_*</code> and browser markup rendering           |
| Query ignores ontology lens                   | <code>min_agent/policy.js</code> and <code>min_agent/ontology_repository.js</code>              |
| Ontology write fails with FK/NOT NULL         | <code>ontology_agent/write_preflight*.js</code> and DB ontology schema                          |
| Ontology asks user after mechanical SQL error | <code>ontology_agent/workflow_repair.js</code> and repair budget flow                           |
| Ontology validates but embeddings fail        | <code>ontology_agent/embedding_refresh.js</code> and <code>ontology_embedding_chunks</code> IDs |
| Taxonomy workflow button does nothing         | <code>browser/main_app/taxonomy_workflow_launcher.ts</code> and chat command routing            |
| Taxonomy workflow blocks                      | Semantic Control Kernel workflow state plus owner adapter logs                                  |
| Kernel dialog does not appear                 | host-only MCP bridge, event polling and <code>kernel_event_policy.ts</code>                     |
| Final workflow message missing                | Kernel final notice mirror event and <code>pipeline_agent/workflow_auto_events.js</code>        |

# 12. Non-Negotiable Boundaries

The agents make the system usable, but they do not erase ownership.

- Query Agent: read-only.
- Ontology Agent: read broadly, write only ontology/relation layer.
- Taxonomy Agent: call Kernel tools, do not bypass Kernel dialogs.
- Corpus Builder: owns Corpus DB schema and materialization.
- Normalizer: owns Semantic Release materialization.
- Orchestrator: owns pipeline execution artifacts.
- Kernel: owns governed workflow state, recovery and target identity.

When debugging, do not fix a boundary violation by making an agent more powerful by default. First check which owner should have performed the action and whether the agent should call that owner through the existing Kernel or tool contract.